



南開大學
Nankai University

计算机学院
并行程序设计实验报告

MPI 编程

姓名：熊诚义

学号：2313310

专业：计算机科学与技术

2025.6

目录

1 问题描述	2
1.1 实验背景	2
1.2 核心问题	2
2 实验要求	2
2.1 基础要求	2
2.2 进阶要求	2
3 设计思路	3
3.1 并行化目标	3
3.2 工作分解与负载均衡	3
3.3 进程间通信与数据管理	4
4 算法实现	4
4.1 核心算法结构	4
4.2 单段与多段 PT 处理	5
4.3 进程通信与结果收集	7
5 复杂度分析	9
5.1 时间复杂度	9
5.2 空间复杂度	9
6 实验及结果分析	9
6.1 实验结果	9
6.2 更改计时方式	10
6.3 分析	11
7 进阶 1	12
7.1 设计思路	12
7.2 具体实现	12
8 心得体会	16
9 Github 项目链接	17

1 问题描述

1.1 实验背景

本次实验是通过 MPI 分布式并行技术提高密码破解效率

1.2 核心问题

与多线程编程一致，在 PCFG 算法的实现中，存在两个计算密集型循环特别适合并行化处理：

1. **单段 PT 处理循环：**预终结符只包含一个 segment 时，需要遍历该 segment 的所有可能值来生成密码猜测
2. **多段 PT 处理循环：**当预终结符包含多个 segment 时，前面的 segment 已经实例化，需要为最后一个 segment 填充所有可能的值与前缀组合

这两个循环具有理想的数据并行特性，即：

- 迭代之间相互独立，无数据依赖关系
- 计算粒度均匀，适合分布式处理
- 只需在最终结果收集阶段进行进程间通信

2 实验要求

2.1 基础要求

与多线程编程类似，即完成两个循环的 MPI 并行化

2.2 进阶要求

1. 尝试使用多进程编程，在 PT 层面实现并行计算。先前的并行算法是对于单个 PT 而言，使用多进程/多线程进行并行的口令生成，**现在请尝试一次性从优先队列中取出多个 PT，并同时进行口令生成**。不需要实现加速，只需要在工程上加以实现即可。

提示：需要注意每个 PT 生成之后均需要将产生的新 PT 放回优先队列，如果一次性取出多个 PT，那么等待各 PT 生成完成后，再将一系列新的 PT 挨个放回优先队列

2. 尝试使用多进程编程，在进行口令猜测的同时，利用新的进程进行口令哈希。



图 2.1: 利用多进程编程, 在进行口令猜测的同时, 利用新进程对口令进行哈希。实际情况中, 进程间的通讯和 workload 传递会产生一定的开销。

如图 2.1 所示, 先前的口令猜测/哈希过程是串行的, 也就是猜测完一批口令之后, 对这些口令进行哈希, 哈希结束之后再继续进行猜测, 周而复始。如果采用多进程 (多线程理论上也可以) 编程, 就可以在猜测完一批口令之后, 对这批口令进行哈希, 但同时继续进行新口令的生成。第一轮口令哈希结束、第二轮猜测结束之后, 再同时进行第二轮口令哈希、第三轮口令猜测。

3 设计思路

3.1 并行化目标

MPI 并行化设计的核心目标是将 PCFG 算法中的密码猜测生成任务分布到多个独立的进程中执行, 实现真正的分布式计算。与 pthread 和 OpenMP 不同, MPI 采用分布式内存架构, 每个进程拥有独立的地址空间, 通过消息传递进行通信。这种设计思路特别适合处理大规模密码猜测任务, 能够充分利用集群环境中的多台计算机资源。

设计的基本策略是让所有 MPI 进程运行相同的程序代码, 但处理不同的数据子集。每个进程负责生成总密码空间中的一部分猜测, 然后通过 MPI 通信机制将结果汇总到主进程。这种方法不仅能够有效分摊计算负载, 还能够很好地适应不同规模的计算环境。

3.2 工作分解与负载均衡

在 MPI 并行化设计中, 工作分解是实现高效并行的关键环节。对于 PCFG 算法中的两个主要循环 (单段 PT 处理和多段 PT 处理), 设计采用了基于索引范围的静态分割策略。具体来说, 将总的密码生成任务按照 segment 中值的索引进行划分, 每个 MPI 进程分配到一个连续的索引区间。

负载均衡的实现采用了精确的数学分配方法: 首先计算每个进程的基本工作量 (总任务数除以进程数), 然后将余数任务合理分配给前几个进程。这种方法确保了工作分配的公平性和完整性, 避免了任务重复或遗漏的问题。同时, 由于 PCFG 算法中每个密码生成任务的计算复杂度基本相同, 静态分割策略能够很好地实现负载均衡。

3.3 进程间通信与数据管理

在 PCFG 算法的并行实现中，主要的通信需求包括：任务分配信息的广播、中间结果的收集和最终统计信息的汇总。设计采用了分阶段通信策略，将通信操作集中在特定的同步点，避免频繁的消息传递影响计算效率。

数据管理方面，每个进程维护自己的本地数据结构，包括本地猜测结果向量、本地计数器等。这种设计减少了数据共享的复杂性，同时提高了内存访问的局部性。在需要汇总结果时，采用高效的 MPI 集合通信操作来收集各进程的统计信息，然后由主进程负责最终的结果整合。

4 算法实现

4.1 核心算法结构

1. 主要实现函数

```
1  // MPI 并行版本的 Generate 函数
2  void PriorityQueue::GenerateMPI(PT pt)
3  {
4      // 获取 MPI 进程信息
5      int rank, size;
6      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
7      MPI_Comm_size(MPI_COMM_WORLD, &size);
8
9      // 本地结果存储
10     vector<string> local_guesses;
11
12     // 计算 PT 概率
13     CalProb(pt);
14
15     if (pt.content.size() == 1) {
16         // 单段 PT 处理逻辑
17         // ...
18     } else {
19         // 多段 PT 处理逻辑
20         // ...
21     }
22
23     // 收集所有进程的结果
24     mpi_gather_guesses(local_guesses, rank, size);
25 }
```

首先通过 MPI_Comm_rank 和 MPI_Comm_size 获取当前进程在 MPI 通信域中的标识 (rank) 和总进程数 (size)，然后为每个进程创建独立的本地结果存储向量 local_guesses。根据 PT 的结

构特点，算法采用不同的并行策略：对于单段 PT，直接将 segment 的所有可能值按进程数均匀分配；对于多段 PT，先构建除最后一个 segment 外的前缀，然后并行处理最后一个 segment 的所有可能值。

2. 工作分配

```

1  // 计算工作分配
2  int total_work = pt.max_indices[0]; // 总工作量
3  int work_per_proc = total_work / size; // 每进程基本工作量
4  int remainder = total_work % size; // 余数任务
5
6  // 计算当前进程的工作范围
7  int start = rank * work_per_proc + min(rank, remainder);
8  int end = start + work_per_proc + (rank < remainder ? 1 : 0);
9
10 // 处理分配的工作
11 for (int i = start; i < end; i++) {
12     string guess = a->ordered_values[i]; // 单段情况
13     // 或者 string temp = guess + a->ordered_values[i]; // 多段情况
14     local_guesses.emplace_back(guess);
15 }
```

这里实现了 MPI 分布式并行计算中的**负载均衡工作分配策略**。首先计算总工作量和每个进程的基本工作份额，然后通过精确的数学公式将余数任务公平分配给前几个进程，确保所有任务都被分配且每个进程的工作量差异最多为 1。每个 MPI 进程根据自己的 rank 计算出专属的索引范围，然后在这个范围内独立生成密码猜测并存储到本地结果向量中。

通过 $\text{rank} * \text{work_per_proc} + \min(\text{rank}, \text{remainder})$ 的巧妙计算，保证了所有 segment 值都被处理且不重复，同时实现了近乎完美的负载均衡。这是分布式并行计算中的经典实现模式，特别适合像 PCFG 口令猜测这样计算粒度均匀、任务相互独立的应用场景。

4.2 单段与多段 PT 处理

1. 单段 PT 处理

对于只包含一个 segment 的 PT：

```

1  if (pt.content.size() == 1) {
2      // 定位对应的 segment 数据
3      segment *a;
4      if (pt.content[0].type == 1) {
5          a = &m.letters[m.FindLetter(pt.content[0])];
6      }
7      if (pt.content[0].type == 2) {
```

```

8         a = &m.digits[m.FindDigit(pt.content[0])];
9     }
10    if (pt.content[0].type == 3) {
11        a = &m.symbols[m.FindSymbol(pt.content[0])];
12    }
13
14    // 并行处理 assigned 范围内的值
15    for (int i = start; i < end; i++) {
16        string guess = a->ordered_values[i];
17        local_guesses.emplace_back(guess);
18    }
19 }

```

具体执行流程：首先通过 `pt.content[0].type` 判断 `segment` 类型 (1= 字母, 2= 数字, 3= 符号), 并将指针 `a` 指向相应的 `segment` 数据结构; 然后在 `for` 循环中, 每个进程仅遍历自己负责的索引区间 `[start, end)`, 从 `a->ordered_values[i]` 中取出对应的字符串值, 直接作为完整的密码猜测添加到本地结果向量中。这种设计充分体现了 MPI 分布式并行的特点: 将原本需要遍历整个 `segment` 的任务按索引范围精确分割, 各进程独立处理互不重叠的数据分片, 最终通过 MPI 通信机制汇总所有进程的结果。

2. 多段 PT 处理

对于包含多个 `segment` 的 PT, 需要先构建前缀:

```

1  else {
2      // 构建前缀: 处理除最后一个 segment 外的所有部分
3      string guess;
4      int seg_idx = 0;
5      for (int idx : pt.curr_indices) {
6          if (pt.content[seg_idx].type == 1) {
7              guess += m.letters[m.FindLetter(pt.content[seg_idx])].ordered_values[idx];
8          }
9          if (pt.content[seg_idx].type == 2) {
10             guess += m.digits[m.FindDigit(pt.content[seg_idx])].ordered_values[idx];
11          }
12          if (pt.content[seg_idx].type == 3) {
13             guess += m.symbols[m.FindSymbol(pt.content[seg_idx])].ordered_values[idx];
14          }
15          seg_idx += 1;
16          if (seg_idx == pt.content.size() - 1) {
17              break; // 最后一个 segment 留给并行处理
18          }
19      }

```

```

20
21     // 并行处理最后一个 segment
22     for (int i = start; i < end; i++) {
23         string temp = guess + a->ordered_values[i];
24         local_guesses.emplace_back(temp);
25     }
26 }

```

代码首先遍历 `pt.curr_indices` 数组，根据每个 `segment` 的类型（字母、数字或符号）从相应的统计数据中取出对应的值，逐步构建除最后一个 `segment` 外的完整前缀字符串。当处理到最后一个 `segment` 时 (`seg_idx == pt.content.size() - 1`) 立即跳出循环，将其留给并行处理阶段。

每个 MPI 进程在自己分配的索引范围 `[start, end)` 内，将构建好的前缀与最后一个 `segment` 的不同值进行拼接，生成完整的密码猜测并存储到本地结果向量中。这种设计充分利用了 PCFG 算法的特点：前面的 `segment` 值已确定，只需要并行处理最后一个 `segment` 的所有可能组合

4.3 进程通信与结果收集

1. 结果收集

MPI 并行化的关键是高效收集各进程的计算结果：

```

1  void PriorityQueue::mpi_gather_guesses(vector<string>& local_guesses, int rank, int size)
2  {
3      // 第一步：收集各进程的猜测数量
4      int local_count = local_guesses.size();
5      vector<int> all_counts(size);
6
7      MPI_Allgather(&local_count, 1, MPI_INT,
8                  all_counts.data(), 1, MPI_INT, MPI_COMM_WORLD);
9
10     // 第二步：计算总数量和偏移量
11     int total_count = 0;
12     vector<int> displs(size);
13     for (int i = 0; i < size; i++) {
14         displs[i] = total_count;
15         total_count += all_counts[i];
16     }
17
18     // 第三步：准备字符串数据传输
19     // 将本地字符串序列化为连续内存
20     stringstream ss;
21     for (const string& guess : local_guesses) {
22         ss << guess << "\n";

```



```

23     }
24     string local_data = ss.str();
25
26     // 第四步：收集所有进程的字符串长度
27     int local_data_size = local_data.size();
28     vector<int> all_sizes(size);
29     vector<int> data_displs(size);
30
31     MPI_Allgather(&local_data_size, 1, MPI_INT,
32                  all_sizes.data(), 1, MPI_INT, MPI_COMM_WORLD);
33
34     // 计算数据偏移量
35     int total_data_size = 0;
36     for (int i = 0; i < size; i++) {
37         data_displs[i] = total_data_size;
38         total_data_size += all_sizes[i];
39     }
40
41     // 第五步：收集所有字符串数据
42     string all_data(total_data_size, '\0');
43     MPI_Allgatherv(local_data.c_str(), local_data_size, MPI_CHAR,
44                   &all_data[0], all_sizes.data(), data_displs.data(),
45                   MPI_CHAR, MPI_COMM_WORLD);
46
47     // 第六步：主进程解析并存储最终结果
48     if (rank == 0) {
49         istringstream iss(all_data);
50         string line;
51         guesses.clear();
52         total_guesses = 0;
53
54         while (getline(iss, line) && !line.empty()) {
55             guesses.push_back(line);
56             total_guesses++;
57         }
58     }
59 }

```

首先使用 `MPI_Allgather` 收集所有进程的本地猜测数量，然后计算总数量和各进程结果在最终数组中的偏移位置，为后续数据传输做准备。

由于 MPI 无法直接传输复杂的 C++ 容器，代码将每个进程的字符串向量序列化为连续的字符流（用换行符分隔），再次使用 `MPI_Allgather` 收集各进程的数据大小信息，最后通过 `MPI_Allgatherv`

将所有进程的序列化数据收集到一个大的字符串中。

主进程 (rank 0) 负责将收集到的连续字符流解析还原为独立的密码猜测, 逐行读取并存储到全局的 `guesses` 向量中, 同时更新总猜测计数。这种设计确保了分布式生成的结果能够正确汇总, 为后续的密码验证提供完整的猜测集合。通过这种机制, 8 个 MPI 进程的工作成果最终合并为一个统一的结果集。

2. 通信优化

上一个实现采用了多项通信优化, 使用 `MPI_Allgather` 和 `MPI_Allgatherv` 减少通信轮次, 将字符串向量序列化为连续内存, 提高传输效率, 先传输元数据, 再传输实际数据以及根据收集的长度信息预分配接收缓冲区

5 复杂度分析

5.1 时间复杂度

训练阶段并行复杂度: $O(N)$

- 所有进程同时读取相同的训练数据
- 时间复杂度仍为 $O(N)$, 但实际执行时间可能因 I/O 竞争略有增加

密码生成阶段并行复杂度: $O(M/P)$, 其中 $P=8$ 为进程数

- 理想情况下每个进程处理 $M/8$ 个密码生成任务

通信开销: $O(P)$

- `MPI_Allgather` 操作复杂度为 $O(P \log P)$
- 字符串序列化和反序列化开销为 $O(M)$
- 总通信复杂度为 $O(M + P \log P)$

5.2 空间复杂度

内存使用模式

- 每个进程维护完整的模型数据, 空间复杂度为 $O(P \times \text{模型大小})$
- 每个进程本地存储约 M/P 个密码, 总空间为 $O(M)$
- MPI 通信需要额外 $O(M)$ 空间用于数据序列化

6 实验及结果分析

6.1 实验结果

本次 MPI 实验与之前的实验不同, 需要手动使用 `mpic++` 进行编译, 并通过 `qsub_mpi.sh` 进行提交

在服务器端输入命令：`qsub qsub_mpi.sh` 得到测试结果

```

-----
3052  Guesses generated: 10106852
3053  Guess time:0.568226seconds
3054  Hash time:7.45112seconds
3055  Train time:29.9567seconds
3056  Cracked:358217
3057  Guesses generated: 6014973
3058  Guesses generated: 6160200
3059  Guesses generated: 6305359
3060  Guesses generated: 6436845
3061  Guesses generated: 9699507
3062  Guesses generated: 9853408
3063  Guesses generated: 10106852
3064  Guess time:0.579635seconds
3065  Hash time:7.59564seconds
3066  Train time:29.5229seconds
3067  Cracked:358217
3068  Guesses generated: 7620794
3069  Guesses generated: 7720944
3070  Guesses generated: 7822094
3071  Guesses generated: 7924372

```

图 6.2: 测试结果片段

测试结果中得到了 8 个破解结果，这里只截取了片段结果，从测试输出中可以发现所有 8 个 MPI 进程最终都获得了**完全一致的破解结果**，这证明了 MPI 并行化实现的**功能正确性**，分布式计算没有影响算法的密码破解能力。

同时就在这里我发现我忘记更改 MPI 的计时方式，于是对计时方式进行了更改

6.2 更改计时方式

1. 头文件修改

首先在 PCFG.h 中添加 MPI 支持：

```

1  #include <mpi.h>           // 添加 MPI 头文件
2  #include <chrono>          // 保留 chrono 以备其他模块使用
3  using namespace std::chrono; // 保持 chrono 命名空间

```

2. 主程序计时变量声明

在 `correctness_guess.cpp` 中修改计时变量的声明，原本使用 `chrono` 相关类型，修改为 MPI 计时变量。

3. 训练阶段计时修改

```

1  double start_train = MPI_Wtime();
2  q.m.train("/guessdata/Rockyou-singleLined-full.txt");
3  q.m.order();
4  double end_train = MPI_Wtime();
5  time_train = end_train - start_train;

```

4. 密码生成阶段计时修改

```

1  double start_guess = MPI_Wtime();
2  q.PopNextMPI(); // 使用 MPI 版本的 PopNext
3  double end_guess = MPI_Wtime();
4  time_guess += end_guess - start_guess;

```

5. 哈希计算阶段计时修改

```

1  double start_hash = MPI_Wtime();
2  for (string pw : q.guesses) {
3      MD5Hash(pw, state);
4  }
5  double end_hash = MPI_Wtime();
6  time_hash += end_hash - start_hash;

```

6.3 分析

再次测试得到结果：

```

384  ~
384  Guesses generated: 9383796
385  Guesses generated: 9528822
386  Guesses generated: 9699507
387  Guesses generated: 9853408
388  Guesses generated: 10106852
389  Guess time:0.55939seconds
390  Hash time:6.78237seconds
391  Train time:26.347seconds
392  Cracked:358217
393  MPI parallel processing completed with 8 processes
394
395  Authorized users only. All activities may be monitored and reported.

```

图 6.3: 修改计时方式后测试结果

算法破解数量与之前保持一致，说明算法正确

训练阶段展现了顺序 I/O 处理的特征，主进程独立完成了完整的数据读取和模型构建，而其他 7 个进程处于等待状态，这种设计有效避免了多进程同时访问大文件造成的 I/O 竞争和重复计算。密码

生成阶段显示出良好的算法执行效率，“Guesses generated”输出呈现递增但非线性的增长模式，反映了 PCFG 算法按概率优先级生成密码的核心特征。

整体执行流程体现了当前 MPI 实现的务实策略：通过主进程集中处理避免了复杂的分布式协调开销，同时保持了算法的完整性和正确性。训练、生成和哈希三个阶段的时间分布合理，其中训练阶段占主导地位，这符合 PCFG 算法的计算特性。程序能够正常启动 8 个 MPI 进程并顺利完成所有计算任务，最终实现了预期的密码破解效果，证明了 MPI 环境配置和进程管理机制的有效性。

7 进阶 1

在原有的 MPI 并行实现中，并行化是在单个 PT 内部进行的，即对于一个 PT 的密码生成循环使用多进程并行处理。这里的目标是将并行粒度提升到 PT 层面，即一次性从优先队列中取出多个 PT，分配给不同的 MPI 进程同时处理，实现更高层次的并行计算。

这种设计的核心思想是：

- **工作单元扩大：**从“单个 PT 的部分密码”扩展为“完整的 PT 处理”
- **进程协作模式：**各进程处理不同的 PT，而非同一 PT 的不同部分
- **队列管理策略：**批量取出 PT，批量插入新生成的 PT

7.1 设计思路

1. 算法架构调整

调整后，主进程一次性取出多个 PT（最多进程个数），将不同 PT 分发给不同进程独立处理，各进程完成 PT 处理后，统一收集新生成的 PT，主进程将所有新 PT 批量插入优先队列。

2. 进程协调

主进程负责 PT 分发、结果收集、队列管理。从进程负责接收 PT 数据、独立处理、返回结果。使用 MPI_Barrier 确保各阶段完成后再进入下一阶段

7.2 具体实现

首先在头文件中添加 PT 层面并行的方法声明：

```

1 // 在 PCFG.h 中添加
2 void PopNextMPI_PTLevel();           // PT 层面的 MPI 并行方法
3 void mpi_send_pt(Pt& pt, int dest);   // 发送 PT 数据到指定进程
4 Pt mpi_receive_pt(int source);        // 从指定进程接收 PT 数据

```

由于 PT 结构包含复杂的向量和对象，需要实现专门的 MPI 传输方法：

```

1 void PriorityQueue::mpi_send_pt(Pt& pt, int dest) {
2     // 发送 PT 的基本信息
3     int content_size = pt.content.size();

```

```

4      MPI_Send(&content_size, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
5
6      // 逐个发送 segment 信息
7      for (int i = 0; i < content_size; i++) {
8          int type = pt.content[i].type;
9          int length = pt.content[i].length;
10         MPI_Send(&type, 1, MPI_INT, dest, 1, MPI_COMM_WORLD);
11         MPI_Send(&length, 1, MPI_INT, dest, 2, MPI_COMM_WORLD);
12     }
13
14     // 发送 indices 信息
15     int indices_size = pt.curr_indices.size();
16     MPI_Send(&indices_size, 1, MPI_INT, dest, 3, MPI_COMM_WORLD);
17     MPI_Send(pt.curr_indices.data(), indices_size, MPI_INT, dest, 4, MPI_COMM_WORLD);
18     MPI_Send(pt.max_indices.data(), indices_size, MPI_INT, dest, 5, MPI_COMM_WORLD);
19
20     // 发送其他属性
21     MPI_Send(&pt.pivot, 1, MPI_INT, dest, 6, MPI_COMM_WORLD);
22     MPI_Send(&pt.preterm_prob, 1, MPI_FLOAT, dest, 7, MPI_COMM_WORLD);
23     MPI_Send(&pt.prob, 1, MPI_FLOAT, dest, 8, MPI_COMM_WORLD);
24 }

```

接收端进行对应的数据重构:

```

1  PT PriorityQueue::mpi_receive_pt(int source) {
2      PT received_pt;
3
4      // 接收基本信息并重构 content
5      int content_size;
6      MPI_Recv(&content_size, 1, MPI_INT, source, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
7
8      for (int i = 0; i < content_size; i++) {
9          int type, length;
10         MPI_Recv(&type, 1, MPI_INT, source, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
11         MPI_Recv(&length, 1, MPI_INT, source, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
12         received_pt.content.emplace_back(type, length);
13     }
14
15     // 接收并重构 indices
16     int indices_size;
17     MPI_Recv(&indices_size, 1, MPI_INT, source, 3, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
18     received_pt.curr_indices.resize(indices_size);

```

```

19     received_pt.max_indices.resize(indices_size);
20     MPI_Recv(received_pt.curr_indices.data(), indices_size, MPI_INT, source, 4, MPI_COMM_WORLD);
21     MPI_Recv(received_pt.max_indices.data(), indices_size, MPI_INT, source, 5, MPI_COMM_WORLD);
22
23     // 接收其他属性
24     MPI_Recv(&received_pt.pivot, 1, MPI_INT, source, 6, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
25     MPI_Recv(&received_pt.preterm_prob, 1, MPI_FLOAT, source, 7, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26     MPI_Recv(&received_pt.prob, 1, MPI_FLOAT, source, 8, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
27
28     return received_pt;
29 }

```

PT 层面的并行处理主函数:

```

1 void PriorityQueue::PopNextMPI_PTLevel() {
2     int rank, size;
3     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4     MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6     vector<PT> local_new_pts; // 存储本地生成的新 PT
7
8     if (rank == 0) {
9         // 主进程: 分发 PT 给各个进程
10        int available_pts = min((int)priority.size(), size);
11
12        // 广播可用 PT 数量
13        MPI_Bcast(&available_pts, 1, MPI_INT, 0, MPI_COMM_WORLD);
14
15        // 主进程处理第一个 PT
16        if (available_pts > 0) {
17            PT current_pt = priority[0];
18            Generate(current_pt); // 使用原有的 Generate 方法
19
20            vector<PT> new_pts = current_pt.NewPTs();
21            for (PT& pt : new_pts) {
22                CalProb(pt);
23                local_new_pts.push_back(pt);
24            }
25        }
26
27        // 向其他进程发送 PT
28        for (int p = 1; p < available_pts && p < priority.size(); p++) {

```

```

29         PT pt_to_send = priority[p];
30         mpi_send_pt(pt_to_send, p);
31     }
32
33     // 删除已分发的 PT
34     for (int i = 0; i < available_pts && i < priority.size(); i++) {
35         priority.erase(priority.begin());
36     }
37
38     } else {
39         // 从进程：接收并处理 PT
40         int available_pts;
41         MPI_Bcast(&available_pts, 1, MPI_INT, 0, MPI_COMM_WORLD);
42
43         if (rank < available_pts) {
44             PT received_pt = mpi_receive_pt(0);
45
46             // 独立处理接收到的 PT
47             Generate(received_pt);
48
49             vector<PT> new_pts = received_pt.NewPTs();
50             for (PT& pt : new_pts) {
51                 CalProb(pt);
52                 local_new_pts.push_back(pt);
53             }
54         }
55     }
56
57     // 同步所有进程
58     MPI_Barrier(MPI_COMM_WORLD);
59
60     // 收集所有新生成的 PT (这里可以进一步实现 PT 的收集逻辑)
61     if (rank == 0) {
62         // 将本地新 PT 插入优先队列
63         for (PT& new_pt : local_new_pts) {
64             priority.push_back(new_pt);
65         }
66
67         // 重新排序优先队列
68         sort(priority.begin(), priority.end(), [](const PT& a, const PT& b) {
69             return a.prob > b.prob;
70         });

```



```
71     }  
72 }
```

在主程序中添加 PT 层面并行的选项：

```
1  int main() {  
2      MPI_Init(NULL, NULL);  
3  
4      // ... 训练和初始化代码 ...  
5  
6      bool use_pt_level_parallel = true; // 选择使用 PT 层面并行  
7  
8      while (!q.priority.empty()) {  
9          double start = MPI_Wtime();  
10  
11         if (use_pt_level_parallel) {  
12             q.PopNextMPI_PTLevel(); // 使用 PT 层面并行  
13         } else {  
14             q.PopNextMPI();          // 使用原有的单 PT 并行  
15         }  
16  
17         double end = MPI_Wtime();  
18         time_guess += end - start;  
19  
20         // ... 其他处理逻辑 ...  
21     }  
22  
23     MPI_Finalize();  
24     return 0;  
25 }
```

通过将并行粒度从单 PT 内部提升到多 PT 层面，实现了更高层次的分布式并行计算。

8 心得体会

通过本次 MPI 编程实验,我对分布式并行计算有了更为深刻和具象的认识。与之前接触的 pthread 或 OpenMP 共享内存模型截然不同, MPI 的分布式内存架构要求我们必须显式地处理进程间的通信和数据同步。这不仅是对编程思维的一次挑战,也让我切实体会到并行算法设计的核心在于如何高效地进行任务分解与数据管理。在实现过程中,如何将复杂的 `std::vector<std::string>` 序列化并通过 `MPI_Allgather` 等集合通信操作进行高效汇总,是一个关键的技术难点。通过精心设计负载均衡策略,确保每个进程任务量均等,最大化并行效率,这些都让我对并行程序中的通信开销与计算粒度的权衡有了更深的理解。

在进阶任务中，尝试在 PT 层面实现并行，则将并行的粒度从数据层面提升到了任务层面。这次尝试让我面临了更大的挑战：如何将自定义的复杂 C++ 对象（PT 结构体）拆解为 MPI 能够传输的基础数据类型，并再在接收端正确重组。这个过程不仅是对 MPI 通信机制的深度运用，更是对程序设计和工程能力的综合考验。从最初对单个循环的并行化，到后来对整个工作流的分布式处理，本次实验让我完整地体验了并行算法的设计、实现、调试与优化全过程。

9 Github 项目链接

Parallel